

TALLER COMPARATIVO

JDBC PURO vs SPRING DATA JPA + HIBERNATE

Estudiante:

Jonathan Rivera Suárez

LINK: <https://github.com/jriveras4-cmyk/APP-WEB-PRACTICA-CON-JPA-JONATHAN-RIVERA>

Nivel:

5to “A” Software y Rediseño

Docente:

Ing. Gleiston Guerrero

Comparación de resultados A continuación, se presenta la tabla comparativa entre la implementación con JDBC puro (PreparedStatement) y la implementación con Spring Data JPA + Hibernate, evaluando líneas de código, tiempo de ejecución del listado, facilidad de mantenimiento y prevención de inyección SQL.

Criterio	JDBC puro con PreparedStatement	Spring Data JPA + Hibernate
(1) Líneas de código (repositorio + conexión)	≈ _____ líneas (pendiente de conteo manual sobre el código fuente del proyecto)	≈ _____ líneas (pendiente de conteo manual sobre el código fuente del proyecto)
(2) Tiempo del listado (100 filas)	149.0795 ms (nanoTime) — método listar ()	67.5029 ms (nanoTime) — método findAll()
(3) Facilidad de mantenimiento	El SQL está en cadenas Java: cambios de esquema exigen retocar cada consulta manualmente; el mapeo ResultSet→objeto es código repetitivo (boilerplate).	El mapeo lo hace Hibernate vía anotaciones; los cambios de esquema se reflejan con Flyway/Liquibase y consultas derivadas o @Query; boilerplate mínimo.
(4) Prevención de SQL Injection	¿Se previene siempre y cuando se use PreparedStatement con marcadores? y setXxx(); si un desarrollador concatena cadenas, la vulnerabilidad reaparece. Evidencia práctica: búsqueda SEGURA con PreparedStatement devolvió 1 resultado; búsqueda INSEGURA con payload malicioso (concatenación de cadenas) devolvió 100 resultados, evidenciando la inyección SQL.	Se previene por defecto: Hibernate genera siempre PreparedStatement parametrizado; para @Query también usa marcadores nombrados o posicionales enlazados como parámetros del método, evitando la concatenación directa de cadenas SQL.

